Homework 02 —Database Normalization and RSQLite/SQLite3

Graded

Student

Kalkidan Tadese

Total Points

9.1 / 10 pts

Autograder Score

0.0 / 0.0

Question 2

A — **3.1** / 4 pts

- **– 0 pts** Correct
- **– 0.8 pts** Did not eliminate repeating groups in question 1.
- **– 0.4 pts** Alias columns are not keys.
- **– 0.4 pts** Missing experiments table in NF2.
- ✔ **– 0.4 pts** Did not eliminate repeating group Aliases in question 1.
- **– 0.4 pts** Incomplete description of tables
- **– 0.5 pts** incorrect melt in question 1
- ✔ **– 0.5 pts** missing aliases table
- **– 0.4 pts** 1NF requires a single table.

Question 3

B — **4** / 4 pts

- ✔ **– 0 pts** Correct
- **– 0.2 pts** Incorrect experiment table
- **– 0.4 pts** Non unique values in experiment table
- **– 0.2 pts** Disconnect database
- **– 0.8 pts** No code for writing the dataframe to database
- **– 1 pt** No code for data manipulation
- **– 0.4 pts** Code does not run
- **– 0.2 pts** Incorrect data table

Question 4

C — **2** / 2 pts

- ✔ **– 0 pts** Correct

Autograder Results

This assignment does not have an autograder configured.

Submitted Files

Kalkidan_Tadese_A.txt 1 Comment Download

Download the partA.csv file from Brightspace and answer the following questions:

1. What changes would be required to perform the first normalization?

a. The original table violates the 1NF principle because it contains repeating groups in the last four columns: “Experiment1_name”, “Experiment2_name”, “Experiment1_Value”, and “Experiment2_Value”. To perform the first normalization, we would need to pivot those columns into rows and then drop the numbers. The table needs to continue to meet the other conditions: each cell contains an atomic value and each row is unique.

2. What changes would be required to perform the second normalization?

a. To normalize the table in First Normal Form to Second Normal Form, we would need to separate sets of value that apply to multiple records to store them into distinct tables. For each group of related records, we would need to provide a primary key. Our 1NF table should be split into three separate tables:
(1) a Gene Background table with the information of the Gene Name, Alias, Alias2, Annotation, Chromosome, Start, End, Strand, and a primary key (GeneID),

Instructor | 03/04 at 3:22 am

Missing aliases table, its also a repeating group

(2) an Experiment table with the experiment types from the Experiment1_Name and Experiment2_Name columns, and a primary key (ExpID),

(3) a Data table with experiment values from the Experiment1_Value and Experiment2_Value columns and a primary key (DataID) as well as two foreign keys linking the related Gene Background and Experiment tables with their respective primary keys, GeneID and ExperimentID.

3. What changes would be required to perform the third normalization step?

a. Before performing the third normalization step, we need to confirm our 2NF design meets all its criteria. Upon doing so, we can move from 2NF to 3NF by eliminating fields that are indirectly related to or independent of our established primary keys.

A gene is defined by its locus, a discrete region of one’s genome. It is through the separate process of gene annotation that genes are ascribed their function, name and features. Gene aliases are related to the gene name, but are not specific to its location, as alias can refer to only a part of the gene or a specific transcript. To distinguish a gene’s physical characteristics from its functional attributes that contribute to its naming and annotation, we should split the Gene Background table into two:

(1) a Gene Identity table storing Gene Name, Alias, Alias2, and Annotation columns, while retaining GeneID as its primary key,

(2) a Gene Coordinates table with the Chromosome, Start, End, and Strand values, including a new primary key and GeneID as a foreign key.

Kalkidan_Tadese_C.ipynb Download

In [1]:
```
#install sqlite3 and pandas
import sqlite3
import pandas as pd
```

In [2]:
```
#connect to sqlite database
con = sqlite3.connect("Kalkidan_Tadese_db.sqlite")
c = con.cursor()
# create pointer/cursor object to interact with db
```

In [4]:
```
# query sqlite_master table to retrieve db metadata
Dtables = pd.read_sql("""SELECT *
FROM sqlite_master
WHERE type='table';""", con)
Dtables #display metadata
```

Out [4]:
```
   type           name        tbl_name  rootpage  \
0  table  sqlite_sequence  sqlite_sequence         7
1  table      Experiment       Experiment         2
2  table          Probes           Probes         3
3  table            Data             Data         6

                                         sql
0           CREATE TABLE sqlite_sequence(name,seq)
1  CREATE TABLE Experiment (\nexpid INTEGER PRIMA...
2  CREATE TABLE Probes ( probeid INTEGER PRIMARY ...
3  CREATE TABLE Data ( \ndataid INTEGER PRIMARY K...
```

In [5]:
```
data_tbl = pd.read_sql("""SELECT * FROM Data;""",con)
# query data table, select everything
data_tbl.dtypes # display the datatypes of columns in Data table
```

Out [5]:
```
dataid       int64
expid        int64
probeid      int64
expvalue   float64
dtype: object
```

In [7]:
```
avg_exp = data_tbl.groupby(["probeid", "expid"])["expvalue"].mean()
#group data by probeid and expid, then calculate mean of expvalue
print(avg_exp.head())
#print last few rows
```

```
probeid  expid
0        0       254.025616
         1       195.552900
```

```
1        0        98.990896
         1       166.035622
2        0       247.066305
Name: expvalue, dtype: float64
```

Kalkidan_Tadese_create_db.ipynb

Download

In [1]:

```
!pip install tabulate
# install package, '!' to run shell commands in notebook
```

Requirement already satisfied: tabulate in /opt/conda/lib/python3.11/site-packages (0.9.0

In [2]:

```
#install necessary packages
import pandas as pd
import numpy as np
from tabulate import tabulate

#function to format a dataframe into nicely formatted tables
#headers uses column names as table headers
def print_table(df):
  return print(tabulate(df,headers = 'keys', tablefmt='fancy_grid'))

# Load the dataset; sep specifies the file is tab-separated
df = pd.read_csv("expvalues.txt", sep="\t")

# Display the first few rows of the dataset
print_table(df.head())
```

	Unnamed: 0	Control1	Control2	Control3	Treatment1	Treatme
0	244901_at	229.986	353.599	178.492	112.908	152.918
1	244902_at	171.15	84.4509	41.372	170.173	134.408
2	244903_at	314.978	373.312	52.9087	196.303	237.515
3	244904_at	24.0437	175.316	94.6842	82.7849	18.4364
4	244905_at	15.8692	40.0412	58.7657	341.353	16.8014

In [3]:

```
# pivot dataframe; transform it from wide to long format
# id_vars, list comprehension; store columns that do not start with control or
treatment in list
df2 = df.melt(
  id_vars=[col for col in df.columns if not col.startswith(('Control', 'Treatment'))],
  value_vars=[col for col in df.columns if col.startswith(('Control', 'Treatment'))],
#stores columns that start with treatment or control that we want to convert into
rows
  var_name='expname', value_name='expvalue' # create new column, 'expname' to
store experiment names; 'expvalue; to store expression values

)
```

```
#remove digits at the end of the experiment names
df2['expname'] = df2['expname'].str.replace(r'\d+', '', regex=True)
```

In [4]:

```
print(df2)
#display the transformed dataframe
```

```
    Unnamed: 0   expname   expvalue
0    244901_at   Control  229.985654
1    244902_at   Control  171.149796
2    244903_at   Control  314.977684
3    244904_at   Control   24.043665
4    244905_at   Control   15.869235
...        ...       ...        ...
1573 245159_at Treatment  126.136578
1574 245160_at Treatment   68.410565
1575 245161_at Treatment  242.856839
1576 245162_at Treatment   10.144342
1577 245163_at Treatment  186.989457

[1578 rows x 3 columns]
```

In [5]:

```
df2 = df2.rename(columns = {'Unnamed: 0':'probename'})
#rename unnamed column to probename

print(df2)
#print to check name change
```

```
     probename   expname   expvalue
0    244901_at   Control  229.985654
1    244902_at   Control  171.149796
2    244903_at   Control  314.977684
3    244904_at   Control   24.043665
4    244905_at   Control   15.869235
...        ...       ...        ...
1573 245159_at Treatment  126.136578
1574 245160_at Treatment   68.410565
1575 245161_at Treatment  242.856839
1576 245162_at Treatment   10.144342
1577 245163_at Treatment  186.989457

[1578 rows x 3 columns]
```

In [6]:

```
experiment = df2[['expname']].drop_duplicates().reset_index(drop=True) #remove
duplicate rows from the expname column
#reset_index(drop=True) drop prior index and assign new index
experiment = experiment.reset_index(names="expid") # store reset index in new
column called expid
print(experiment)
```

```
   expid   expname
0      0   Control
```

```
1     1  Treatment
```

In [7]:

```
probes = df2[["probename"]].drop_duplicates() #remove duplicate rows from the probename column
probes = probes.reset_index(names = "probeid") #reset dataframe index and store in a column called probeid

print(probes) #display df to check
```

```
    probeid  probename
0         0  244901_at
1         1  244902_at
2         2  244903_at
3         3  244904_at
4         4  244905_at
..      ...        ...
258     258  245159_at
259     259  245160_at
260     260  245161_at
261     261  245162_at
262     262  245163_at

[263 rows x 2 columns]
```

In [8]:

```
de1 = df2.join(experiment.set_index("expname"), on="expname")
#join to add expid column from the experiment table to original dataframe (df2_
de1 = de1.join(probes.set_index("probename"), on="probename")
#join to add probeid column from the probes table to original dataframe (df2)

#create new column called dataid, with reset index
df_data = de1.reset_index(names = "dataid")
#Select only these columns for the Data table
Data = df_data[["dataid","expid","probeid","expvalue"]]
print(Data)

#print
print(Data.dtypes)
```

```
     dataid  expid  probeid    expvalue
0         0      0        0  229.985654
1         1      0        1  171.149796
2         2      0        2  314.977684
3         3      0        3   24.043665
4         4      0        4   15.869235
...     ...    ...      ...         ...
1573   1573      1      258  126.136578
1574   1574      1      259   68.410565
1575   1575      1      260  242.856839
1576   1576      1      261   10.144342
1577   1577      1      262  186.989457

[1578 rows x 4 columns]
```

```
dataid       int64
expid        int64
probeid      int64
expvalue   float64
dtype: object
```

In [9]:

```
?pd.DataFrame.join
```

```
[0;31mSignature:  [0m
[0mpd [0m [0;34m. [0m [0mDataFrame [0m [0;34m. [0m [0mjoin [0m [0;34m( [0
[0;34m [0m     [0mself [0m [0;34m, [0m [0;34m [0m
[0;34m [0m     [0mother [0m [0;34m: [0m  [0;34m'DataFrame | Series | Iterable[Dat
[0;34m [0m     [0mon [0m [0;34m: [0m  [0;34m'IndexLabel | None' [0m  [0;34m= [
[0;34m [0m     [0mhow [0m [0;34m: [0m  [0;34m'MergeHow' [0m  [0;34m= [0m  [
[0;34m [0m     [0mlsuffix [0m [0;34m: [0m  [0;34m'str' [0m  [0;34m= [0m  [0;34m'
[0;34m [0m     [0mrsuffix [0m [0;34m: [0m  [0;34m'str' [0m  [0;34m= [0m  [0;34m'
[0;34m [0m     [0msort [0m [0;34m: [0m  [0;34m'bool' [0m  [0;34m= [0m  [0;32mF
[0;34m [0m     [0mvalidate [0m [0;34m: [0m  [0;34m'JoinValidate | None' [0m  [0;34
[0;34m [0m [0;34m) [0m  [0;34m-> [0m  [0;34m'DataFrame' [0m [0;34m [0m [0;34
[0;31mDocstring:  [0m
Join columns of another DataFrame.

Join columns with `other` DataFrame either on index or on a key
column. Efficiently join multiple DataFrame objects by index at once by
passing a list.

Parameters
----------
other : DataFrame, Series, or a list containing any combination of them
    Index should be similar to one of the columns in this one. If a
    Series is passed, its name attribute must be set, and that will be
    used as the column name in the resulting joined DataFrame.
on : str, list of str, or array-like, optional
    Column or index level name(s) in the caller to join on the index
    in `other`, otherwise joins index-on-index. If multiple
    values given, the `other` DataFrame must have a MultiIndex. Can
    pass an array as the join key if it is not already contained in
    the calling DataFrame. Like an Excel VLOOKUP operation.
how : {'left', 'right', 'outer', 'inner', 'cross'}, default 'left'
    How to handle the operation of the two objects.

    * left: use calling frame's index (or column if on is specified)
    * right: use `other`'s index.
    * outer: form union of calling frame's index (or column if on is
      specified) with `other`'s index, and sort it lexicographically.
    * inner: form intersection of calling frame's index (or column if
      on is specified) with `other`'s index, preserving the order
      of the calling's one.
    * cross: creates the cartesian product from both frames, preserves the order
      of the left keys.

      .. versionadded:: 1.2.0

lsuffix : str, default ''
```

```
    Suffix to use from left frame's overlapping columns.
rsuffix : str, default ''
    Suffix to use from right frame's overlapping columns.
sort : bool, default False
    Order result DataFrame lexicographically by the join key. If False,
    the order of the join key depends on the join type (how keyword).
validate : str, optional
    If specified, checks if join is of specified type.

    * "one_to_one" or "1:1": check if join keys are unique in both left
      and right datasets.
    * "one_to_many" or "1:m": check if join keys are unique in left dataset.
    * "many_to_one" or "m:1": check if join keys are unique in right dataset.
    * "many_to_many" or "m:m": allowed, but does not result in checks.

    .. versionadded:: 1.5.0

Returns
-------
DataFrame
    A dataframe containing columns from both the caller and `other`.

See Also
--------
DataFrame.merge : For column(s)-on-column(s) operations.

Notes
-----
Parameters `on`, `lsuffix`, and `rsuffix` are not supported when
passing a list of `DataFrame` objects.

Examples
--------
>>> df = pd.DataFrame({'key': ['K0', 'K1', 'K2', 'K3', 'K4', 'K5'],
...                    'A': ['A0', 'A1', 'A2', 'A3', 'A4', 'A5']})

>>> df
  key   A
0  K0  A0
1  K1  A1
2  K2  A2
3  K3  A3
4  K4  A4
5  K5  A5

>>> other = pd.DataFrame({'key': ['K0', 'K1', 'K2'],
...                       'B': ['B0', 'B1', 'B2']})

>>> other
  key   B
0  K0  B0
1  K1  B1
2  K2  B2

Join DataFrames using their indexes.
```

```
>>> df.join(other, lsuffix='_caller', rsuffix='_other')
  key_caller   A key_other    B
0         K0  A0       K0   B0
1         K1  A1       K1   B1
2         K2  A2       K2   B2
3         K3  A3      NaN  NaN
4         K4  A4      NaN  NaN
5         K5  A5      NaN  NaN
```

If we want to join using the key columns, we need to set key to be the index in both `df` and `other`. The joined DataFrame will have key as its index.

```
>>> df.set_index('key').join(other.set_index('key'))
     A    B
key
K0   A0   B0
K1   A1   B1
K2   A2   B2
K3   A3  NaN
K4   A4  NaN
K5   A5  NaN
```

Another option to join using the key columns is to use the `on` parameter. DataFrame.join always uses `other`'s index but we can use any column in `df`. This method preserves the original DataFrame's index in the result.

```
>>> df.join(other.set_index('key'), on='key')
  key   A    B
0  K0  A0   B0
1  K1  A1   B1
2  K2  A2   B2
3  K3  A3  NaN
4  K4  A4  NaN
5  K5  A5  NaN
```

Using non-unique key values shows how they are matched.

```
>>> df = pd.DataFrame({'key': ['K0', 'K1', 'K1', 'K3', 'K0', 'K1'],
...                    'A': ['A0', 'A1', 'A2', 'A3', 'A4', 'A5']})

>>> df
  key   A
0  K0  A0
1  K1  A1
2  K1  A2
3  K3  A3
4  K0  A4
5  K1  A5

>>> df.join(other.set_index('key'), on='key', validate='m:1')
  key   A    B
0  K0  A0   B0
1  K1  A1   B1
2  K1  A2   B1
```

```
3  K3  A3  NaN
4  K0  A4   B0
5  K1  A5   B1
  [0;31mFile:  [0m     /opt/conda/lib/python3.11/site-packages/pandas/core/frame.py
  [0;31mType:  [0m     function
```

In [15]:

```python
import sqlite3

#connect to sqlite database; creates db if it doesn't exist
con = sqlite3.connect("Kalkidan_Tadese_db.sqlite")
c = con.cursor()
# create pointer/cursor object to interact with db

c.execute("DROP TABLE IF EXISTS Experiment")
con.commit() # explicit commit db changes

#create Experiment table if it doesn't exist; provide attributes -- two columns, data types
c.execute("""CREATE TABLE IF NOT EXISTS Experiment (
expid INTEGER PRIMARY KEY, expname TEXT NOT NULL )""")

#load values from experiment dataframe into Experiment table; if table already exists, replace w new data; w/o index
experiment.to_sql("Experiment", con, if_exists='replace', index=False)

# execute sql query to select everything from the Experiment table
c.execute("""SELECT * from Experiment""")

# Fetchall - fetches all records
# as list of tuples
recordstbl1 = c.fetchall()
print(recordstbl1)
```

```
[(0, 'Control'), (1, 'Treatment')]
```

In [16]:

```python
c.execute("DROP TABLE IF EXISTS Probes")
con.commit() # explicit commit changes to db


#create Probes table if it doesn't exist; provide attributes -- two columns, respective attributes
c.execute("""CREATE TABLE IF NOT EXISTS Probes ( probeid INTEGER PRIMARY KEY, probename TEXT NOT NULL )""")

#load values from probes dataframe into Probes table; if table already exists, replace w new data; w/o index
probes.to_sql("Probes", con, if_exists='replace', index=False)

# execute sql query to select everything from the Probes table
c.execute("""select * from Probes""")

# Fetch all records
```

```
recordstbl2 = c.fetchall()
print(recordstbl2)
```

```
[(0, '244901_at'), (1, '244902_at'), (2, '244903_at'), (3, '244904_at'), (4, '244905_at'), (5, '2449
```

In [17]:

```
# execute sql query to drop Data table if it already exists; drop to remove potential conflict w later create Data query
c.execute("DROP TABLE IF EXISTS Data")
con.commit() # explicit commit changes to db

#create Data table if it doesn't exist; provide attributes -- four columns, respective attributes
c.execute("""CREATE TABLE IF NOT EXISTS Data (
dataid INTEGER PRIMARY KEY AUTOINCREMENT,
expid INTEGER,
probeid INTEGER,
expvalue REAL,
FOREIGN KEY (expid) REFERENCES Experiment(expid),
FOREIGN KEY (probeid) REFERENCES Probes(probeid))""")

#load values from Data df into Data table; if table already exists, add data; w/o index
Data.to_sql("Data", con, if_exists='append', index=False)
con.commit() #commit changes to db

c.execute("PRAGMA table_info(Data)") # query information about the Data table -- to confirm table format
print(c.fetchall()) ## Fetchall - fetches all records stored in c and prints them
```

```
[(0, 'dataid', 'INTEGER', 0, None, 1), (1, 'expid', 'INTEGER', 0, None, 0), (2, 'probeid', 'INTEGE
```

In [21]:

```
print(Data.head())  # Preview some rows
```

```
   dataid  expid  probeid    expvalue
0       0      0        0  229.985654
1       1      0        1  171.149796
2       2      0        2  314.977684
3       3      0        3   24.043665
4       4      0        4   15.869235
```

Kalkidan_Tadese_db.sqlite Download

1 Binary file hidden. You can download it using the button above.